Natural Language Processing

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 18

Context-Free Grammars and Constituency Parsing

Syntax

- *Syntax* comes from the Greek *sýntaxis*, meaning “setting out together or arrangement”: refers to the way words are arranged together.
- Focus in this chapter: **context-free grammars** and the **CKY algorithm** for parsing them.
- Context-free grammars are the backbone of many formal models of the syntax of natural language and, also, of computer languages.
- Syntactic parsing is the task of assigning a syntactic structure, a **parse tree**, to a sentence.
- Parse trees can be used in applications such as grammar checking.

Constituency

- **Syntactic constituency** is the idea that groups of words within utterances can act as single units, or **constituents**.
- In a given language, these units form coherent classes that can be shown to behave in similar ways
  - With respect to their internal structure
  - And with respect to other units in the language
- Constituents can range from a single word to larger phrases.
  - Internal structure
    - We can describe an internal structure to the class (Constituents are typically composed of words from different **lexical categories** such as nouns, verbs, adjectives, etc.).
  - External behavior
    - Example, we can say that noun phrases can come before verbs; also in terms of agreements such subject-verb agreement.

Constituency

- Part of developing a grammar involves building an inventory of the constituents in the language.
- Example: a noun phrase is a sequence of words surrounding at least one noun
- It makes sense to the say that the following are all ***noun phrases*** in English...

Harry the Horse	a high-class spot such as Mindy's
the Broadway coppers	the reason he comes into the Hot Box
they	three parties from Brooklyn

- Why?
- One piece of evidence is that they can all precede verbs (not true of each of the individual words that make up a noun phrase).
  - This is external evidence

Context-Free Grammars

- **Context-free grammars** (**CFG**s): a widely used formal system for modeling constituent structure in natural language.
  - Also known as
    - Phrase structure grammars
    - Backus-Naur form
- A CFG consists of
  - A set of **rules** each of which expresses the ways that symbols of the language can be grouped and ordered together, and
  - A **lexicon** of words and symbols.

Context-Free Grammars

- Terminals
  - We'll take these to be words (for now)
- Non-Terminals
  - The constituents in a language
    - Like noun phrase, verb phrase and sentence
  - The non-terminal associated with each word in the lexicon is its lexical category, or part of speech.
- Rules
  - Rules consist of a single non-terminal on the left and any number of terminals and non-terminals on the right.

Example: Some NP Rules

- Here are some rules for noun phrases

$$
\begin{aligned}
NP &\rightarrow Det\ Nominal \\
NP &\rightarrow ProperNoun \\
Nominal &\rightarrow Noun \mid Nominal\ Noun
\end{aligned}
$$

- Together, these describe two kinds of NPs.
  - One that consists of a determiner followed by a nominal
  - And another that says that proper names are NPs.
  - The third rule illustrates two things
    - An explicit disjunction
      - Two kinds of nominals
    - A recursive definition
      - Same non-terminal on the right and left-side of the rule

CFG Definition

- More formally, a **CFG** consists of

N	a set of **non-terminal symbols** (or **variables**)
Σ	a set of **terminal symbols** (disjoint from N)
R	a set of **rules** or productions, each of the form $A \rightarrow \beta$, where A is a non-terminal, β is a string of symbols from the infinite set of strings $(\Sigma \cup N)*$
S	a designated **start symbol**

- **Parsing** is the process of taking a string and a grammar and returning a (multiple?) parse tree(s) for that string.

L_0 Grammar

- This (simplified) grammar gives a number of rules for English sentences:.

Grammar Rules			Examples
S	→	*NP VP*	I + want a morning flight
NP	→	*Pronoun*	I
	|	*Proper-Noun*	Los Angeles
	|	*Det Nominal*	a + flight
Nominal	→	*Nominal Noun*	morning + flight
	|	*Noun*	flights
VP	→	*Verb*	do
	|	*Verb NP*	want + a flight
	|	*Verb NP PP*	leave + Boston + in the morning
	|	*Verb PP*	leaving + on Thursday
PP	→	*Preposition NP*	from + Los Angeles

L_0 Grammar (cont.)

… and the (partial) lexicon:

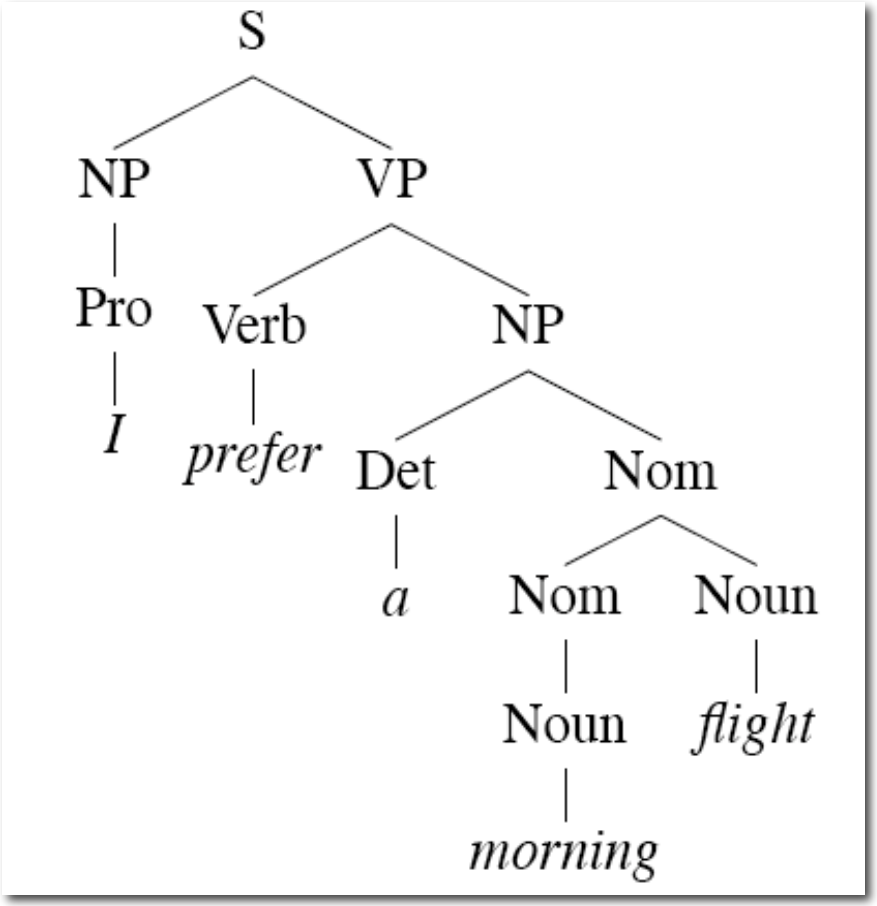
Noun →	*flights* | *flight* | *breeze* | *trip* | *morning*
Verb →	*is* | *prefer* | *like* | *need* | *want* | *fly* | *do*
Adjective →	*cheapest* | *non-stop* | *first* | *latest* | *other* | *direct*
Pronoun →	*me* | *I* | *you* | *it*
Proper-Noun →	*Alaska* | *Baltimore* | *Los Angeles* | *Chicago* | *United* | *American*
Determiner →	*the* | *a* | *an* | *this* | *these* | *that*
Preposition →	*from* | *to* | *on* | *near* | *in*
Conjunction →	*and* | *or* | *but*

Generativity

- A CFG can be thought of in two ways:
  - as a device for generating sentences (synthesis) and
  - as a device for assigning a structure to a given sentence (analysis).
- So they allow to
  - Generate strings in the language
  - Reject strings not in the language
  - Impose structures (trees) on strings in the language

Derivations

- A *derivation* is a sequence of rules applied to a string that *accounts* for that string
  - Covers all the elements in the string.
  - Covers only the elements in the string.



A parse tree for “I prefer a morning flight”

Derivation

- A parse tree can also be represented in a more compact format called bracketed notation.
- Here is the bracketed representation of the previous parse tree for the sentence “I prefer a morning flight”:

```
[S [NP [Pro I]][VP [V prefer][NP [Det a][Nom
[Nom [N morning][N flight]]]]]]
```

- A CFG like that of $\mathbf{L_0}$ defines a formal language.
- Sentences that can be derived by a grammar are in the formal language defined by that grammar, and are called **grammatical sentences**.
- Sentences that cannot be derived by a given formal grammar are not in the language defined by that grammar and are referred to as **ungrammatical**.

Agreement

- By ***agreement***, we have in mind constraints that hold among various constituents that take part in a rule or set of rules
- For example, in English, determiners and the head nouns in NPs have to agree in their number.

This flight

Those flights

*This flights

*Those flight

Problem

- Our earlier NP rules are clearly deficient since they don't capture this constraint
  - *NP* → *Det Nominal*
    - Accepts, and assigns correct structures, to grammatical examples (*this flight*)
    - But its also happy with incorrect examples (*these flight)
  - Such a rule is said to ***overgenerate***.

Verb Phrases

- English *VP*s consist of a head verb along with 0 or more following constituents which we Will call *arguments*.

VP	→	*Verb*	disappear
VP	→	*Verb NP*	prefer a morning flight
VP	→	*Verb NP PP*	leave Boston in the morning
VP	→	*Verb PP*	leaving on Thursday

Subcategorization

- But, even though there are many valid VP rules in English, not all verbs are allowed to participate in all those VP rules.
- We can subcategorize the verbs in a language according to the sets of VP rules that they participate in.
- Modern grammars may have 100s or such classes.

Subcategorization

- Sneeze: John sneezed
- Find: Please find [a flight to NY]$_{NP}$
- Give: Give [me]$_{NP}$[a cheaper fare]$_{NP}$
- Help: Can you help [me]$_{NP}$[with a flight]$_{PP}$
- Prefer: I prefer [to leave earlier]$_{TO\text{-}VP}$
- Told: I was told [United has a flight]$_{S}$
- …

Subcategorization

- *John sneezed the book
- *I prefer United has a flight
- *Give with a flight

- As with agreement phenomena, we need a way to formally express these facts

Why?

- Right now, the various rules for VPs *overgenerate*.
  - They permit the presence of strings containing verbs and arguments that don't go together
  - For example
  - VP -> V NP therefore

    Sneezed the book is a VP since "sneeze" is a verb and "the book" is a valid NP

Possible CFG Solution

- Possible solution for agreement.
- Can use the same trick for all the verb/VP classes.

- SgS -> SgNP SgVP
- PlS -> PlNp PlVP
- SgNP -> SgDet SgNom
- PlNP -> PlDet PlNom
- PlVP -> PlV NP
- SgVP ->SgV Np
- …

CFG Solution for Agreement

- It works and stays within the power of CFGs.
- But it is ugly.
- And it doesn't scale all that well because it explodes the number of rules in our grammar.
- A better approach which goes out of the CFG formalism uses arguments of the non-terminals (**syntactic features**), similar to First-Order Logic.

Syntactic Agreement (Accords): ugly approach

- a. this dog b. *these dog
- a. these dogs b. *this dogs
- a. the dog runs b. *the dog run
- a. the dogs run b. *the dogs runs

S -> NP_SG VP_SG

NP_SG -> Det_SG N_SG

VP_SG -> V_SG

Det_SG -> 'this‘

N_SG -> 'dog‘

V_SG -> 'runs‘

S -> NP_PL VP_PL

NP_PL -> Det_PL N_PL

VP_PL -> V_PL

Det_PL -> 'these'

N_PL -> 'dogs'

V_PL -> 'run'

Syntactic Features

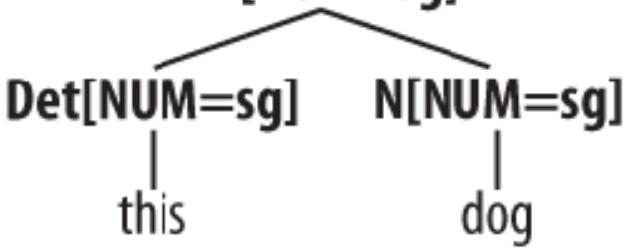
- We add features e.g. **N[NUM=pl]**

```
Det[NUM=sg] -> 'this'
Det[NUM=pl] -> 'these'
N[NUM=sg] -> 'dog'
N[NUM=pl] -> 'dogs'
V[NUM=sg] -> 'runs'
V[NUM=pl] -> 'run‘
```

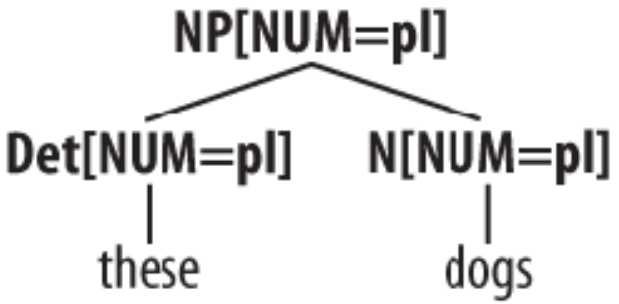
- And variables can also be used:

```
S                -> NP[NUM=?n] VP[NUM=?n]
NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n]
VP[NUM=?n] -> V[NUM=?n]
```

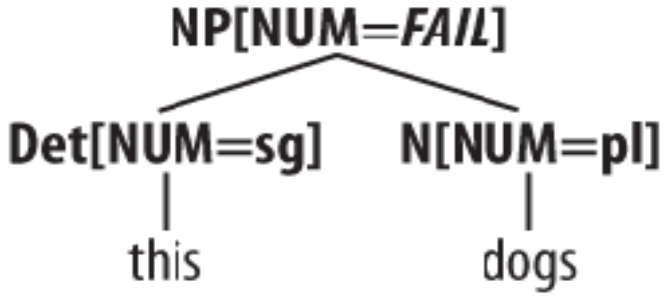
a.


NP[NUM=sg]
Det[NUM=sg]
N[NUM=sg]
this
dog

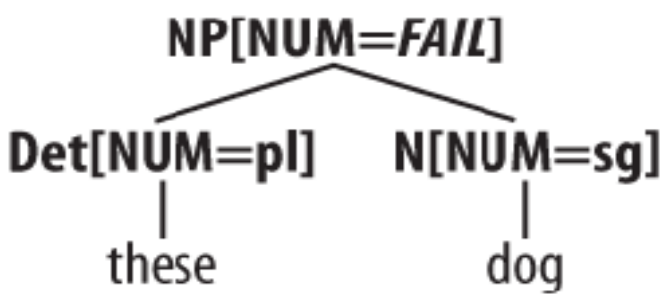
b.

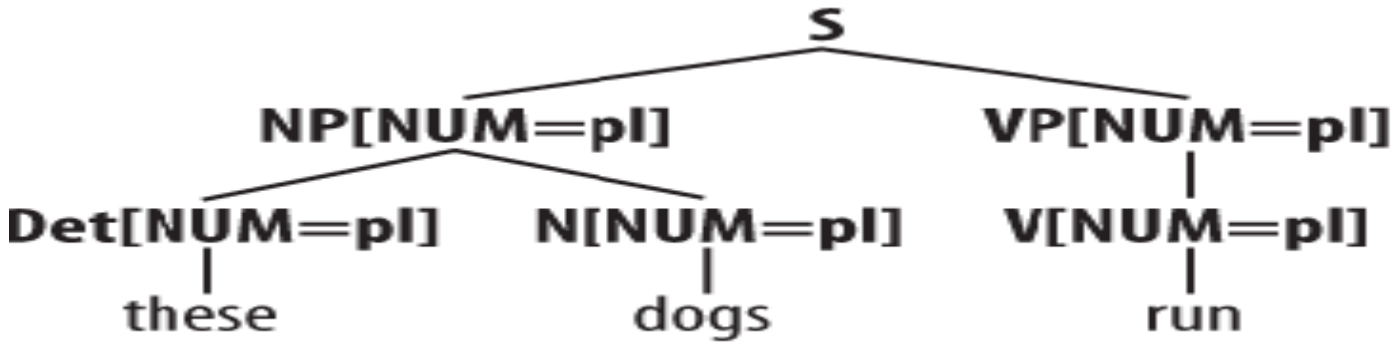

NP[NUM=pl]
Det[NUM=pl]
N[NUM=pl]
these
dogs

a.


NP[NUM=FAIL]
Det[NUM=sg]
N[NUM=pl]
this
dogs

b.


NP[NUM=FAIL]
Det[NUM=pl]
N[NUM=sg]
these
dog


S
NP[NUM=pl]
VP[NUM=pl]
Det[NUM=pl]
N[NUM=pl]
V[NUM=pl]
these
dogs
run

Grammars with Syntactic Features

Det[NUM=sg] -> 'the' | 'some'

Det[NUM=pl] -> 'the' | 'some' | 'several‘

And even better:

Det[NUM=?n] -> 'the' | 'some‘

- Hence the following grammar:

Grammars with Syntactic Features

```
>>> nltk.data.show_cfg('grammars/book_grammars/feat0.fcfg')
% start S
# ###################
# Grammar Productions
# ###################
S -> NP[NUM=?n] VP[NUM=?n]
# NP expansion productions
NP[NUM=sg] -> N[NUM=sg]
NP[NUM=?n] -> PropN[NUM=?n]
NP[NUM=?n] -> Det[NUM=?n] N[NUM=?n]
NP[NUM=pl] -> N[NUM=pl]

# VP expansion productions
VP[TENSE=?t, NUM=?n] -> IV[TENSE=?t, NUM=?n]
VP[TENSE=?t, NUM=?n] -> TV[TENSE=?t, NUM=?n] NP
```

```
# ####################
# Lexical Productions
# ####################
Det[NUM=sg] -> 'this' | 'every'
Det[NUM=pl] -> 'these' | 'all'
Det[NUM=?] -> 'the' | 'some' | 'several'
PropN[NUM=sg]-> 'Kim' | 'Jody'
N[NUM=sg] -> 'dog' | 'girl' | 'car' | 'child'
N[NUM=pl] -> 'dogs' | 'girls' | 'cars' | 'children'
IV[TENSE=pres, NUM=sg] -> 'disappears' | 'walks'
TV[TENSE=pres, NUM=sg] -> 'sees' | 'likes'
IV[TENSE=pres, NUM=pl] -> 'disappear' | 'walk'
TV[TENSE=pres, NUM=pl] -> 'see' | 'like'
IV[TENSE=past] -> 'disappeared' | 'walked'
TV[TENSE=past] -> 'saw' | 'liked'
```

Treebanks

- Treebanks are corpora in which each sentence has been paired with a parse tree (presumably the right one).
- These are generally created
  - By first parsing the collection with an automatic parser
  - And then having human annotators correct each parse as necessary.
- This generally requires detailed annotation guidelines that provide a POS tagset, a grammar and instructions for how to deal with particular grammatical constructions.

Penn Treebank

- Penn TreeBank is a widely used treebank.

- Most well known is the Wall Street Journal section of the Penn TreeBank.
  - 1 M words from the 1987-1989 Wall Street Journal.

```
( (S (`` ``)
    (S-TPC-2
      (NP-SBJ-1 (PRP We) )
      (VP (MD would)
        (VP (VB have)
          (S
            (NP-SBJ (-NONE- *-1) )
            (VP (TO to)
              (VP (VB wait)
                (SBAR-TMP (IN until)
                  (S
                    (NP-SBJ (PRP we) )
                    (VP (VBP have)
                      (VP (VBN collected)
                        (PP-CLR (IN on)
                          (NP (DT those)(NNS assets)))))))))))))
    (, ,) (`` ``)
    (NP-SBJ (PRP he) )
    (VP (VBD said)
      (S (-NONE- *T*-2) ))
    (. .) ))
```

Treebank Grammars

- Treebanks implicitly define a grammar for the language covered in the treebank.
- Simply take the local rules that make up the sub-trees in all the trees in the collection and you have a grammar.
- Not complete, but if you have decent size corpus, you'll have a grammar with decent coverage.

Treebank Grammars

- Such grammars tend to be very flat due to the fact that they tend to avoid recursion.
  - To ease the annotators' burden
- For example, the Penn Treebank has 4500 different rules for VPs. Among them...

```
VP → VBD PP
VP → VBD PP PP
VP → VBD PP PP PP
VP → VBD PP PP PP PP
```

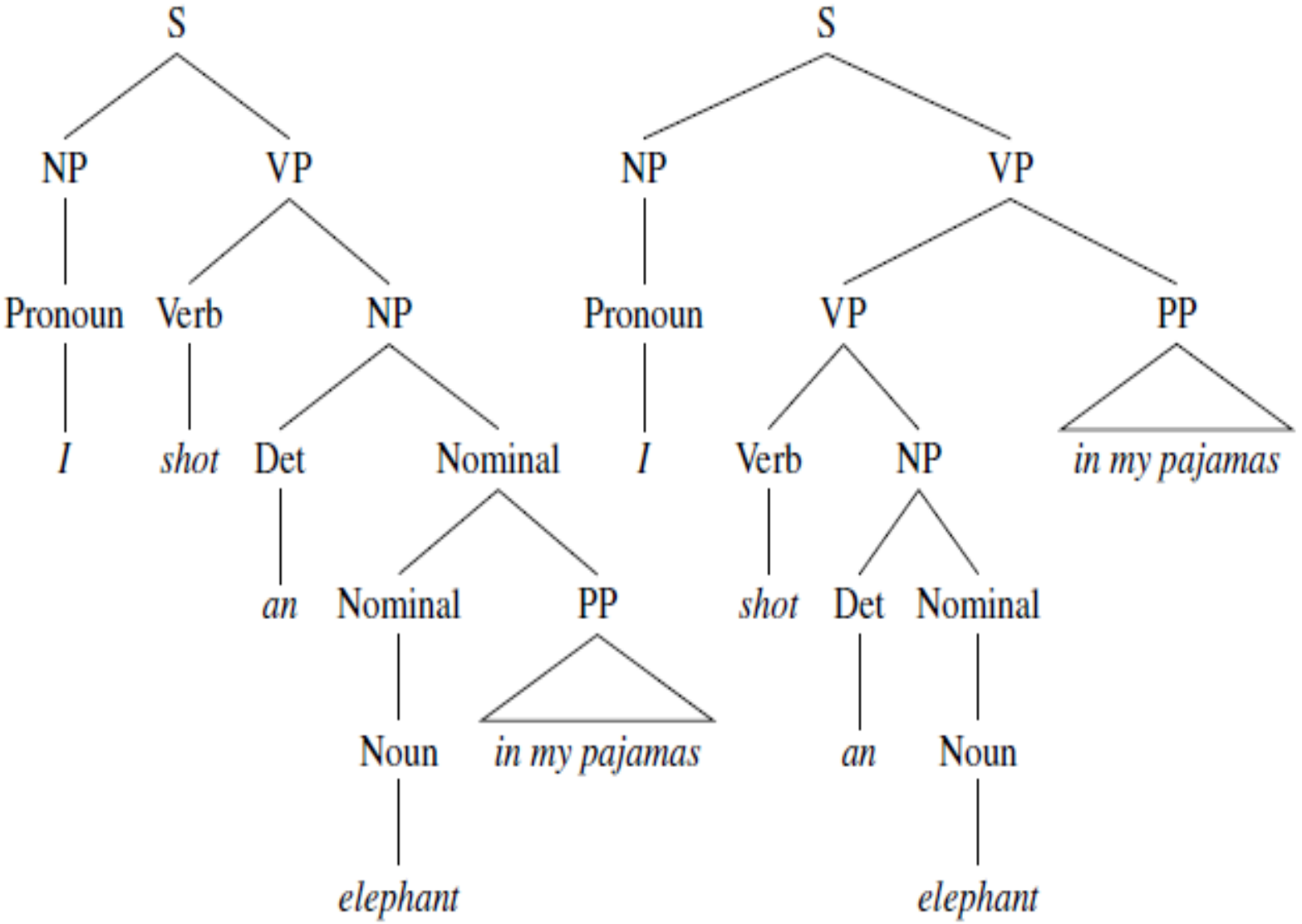
Compared to...

VP -> VP PP
VP -> VBD PP

Ambiguity

- **Ambiguity**: most serious problem faced by syntactic parsers.
  - Part-of-speech ambiguity.
  - Structural ambiguity.
- Words are ambiguous—have more than one possible POS—and the goal is to find the correct tag for the situation.
- For example, *book* can be
  - a verb (*book* that flight) or a noun (hand me that *book*)
  - *That* can be a determiner (Does *that* flight serve dinner) or a complementizer (I thought *that* your flight was earlier).
- The goal of **POS-tagging** is to resolve these **resolution ambiguities**, choosing the proper tag for the context. (See Chap. 17)
- **Structural ambiguity** occurs when the grammar can assign more than one parse to a sentence.

Structural Ambiguity



Structural Ambiguity

- Two common kinds of structural ambiguity:
  - **Attachment ambiguity** and
  - **Coordination ambiguity**.
- A sentence has an attachment ambiguity if a particular constituent can be attached to the parse tree at more than one place.
- Example of PP-attachment ambiguity: *I shot an elephant in my pyjamas*
  - Preposition phrase as part of the NP or as part of the VP.
- Various kinds of adverbial phrases are also subject to this kind of ambiguity. For instance,
  - *We saw the Eiffel Tower flying to Paris*
  - The gerundive-VP (*flying to Paris)*!

Structural Ambiguity

- In **coordination ambiguity** phrases can be conjoined by a conjunction like *and*.
- For example, the phrase *old men and women* can be bracketed as
  - *[old [men and women]]* or as
  - *[old men] and [women]*
- There are many grammatically correct but semantically unreasonable parses for naturally occurring sentences: A challenging problem that affects all parsers.
- The CKY algorithm is designed to efficiently handle structural ambiguities.
- We'll also see that we can augment CKY with neural methods to choose a single correct parse by syntactic disambiguation.

Cocke-Kasami-Younger (CKY) Parsing Alg.

- The dynamic programming approach systematically fills in a table of solutions to subproblems.
- The complete table has the solution to all the subproblems needed to solve the problem as a whole.
- In the case of syntactic parsing, these subproblems represent parse trees for all the constituents detected in the input.
- Dynamic programming provides a powerful framework for addressing the problems caused by ambiguity in grammars.
- Dynamic programming advantage arises from context-free nature of the grammar rules: *once a constituent has been discovered in a segment of the input we can record its presence and make it available for use in any subsequent derivation that might require it*.

  ➔ Time and storage efficiencies since subtrees can be looked up in a table, not reanalyzed.

Conversion to Chomsky Normal Form

- Recall: grammars in CNF are restricted to rules of the form

  $A \rightarrow B\ C$ (B and C non-terminals) or $A \rightarrow w$ (w a terminal)

- There are 3 situations to address in any generic grammar:
  1. rules that mix terminals with non-terminals on the RHS,
  2. rules that have a single non-terminal on the RHS, and
  3. rules in which the length of the RHS is greater than 2.
- Case 1: introduce a new dummy non-terminal that covers only the original terminal:
  - *INF-VP* → *to VP*

    would be replaced by the two rules

    INF-VP → *TO VP* and *TO* → *to*

Conversion to Chomsky Normal Form

- Case 2: Rules with a single non-terminal on the right are called **unit productions**.
- Unit productions can be eliminated by rewriting the RHS of the original rules with the RHS of all the non-unit production rules that they ultimately lead to.
- More formally, if $A \overset{*}{\Rightarrow} B$ by a chain of one or more unit productions and $B \rightarrow \gamma$ is a non-unit production in the grammar, then add $A \rightarrow \gamma$ for each such rule in the grammar and discard all the intervening unit productions.
- This can lead to a substantial flattening of the grammar and a consequent promotion of terminals to fairly high levels in the resulting trees.

Conversion to Chomsky Normal Form

- Case 3: Rules with RHS longer than 2 are normalized through the introduction of new NTs that spread the longer sequences over several new rules.
- Formally, for a rule like $A \rightarrow B\ C\ g$

  replace the leftmost pair of non-terminals with a new NT and introduce a new production, resulting in the new rules:

  $A \rightarrow X1\ g$

  $X1 \rightarrow B\ C$
- In the case of longer RHSs, simply iterate this process until the offending rule has been replaced by rules of length 2.
- In the current grammar, the rule

  $S \rightarrow Aux\ NP\ VP$

  would be replaced by the two rules

  $S \rightarrow X1\ VP$ and $X1 \rightarrow Aux\ NP$

Conversion to Chomsky Normal Form

- The entire conversion process can be summarized as follows:
  1. Copy all conforming rules to the new grammar unchanged.
  2. Convert terminals within rules to dummy non-terminals.
  3. Convert unit productions.
  4. Make all rules binary and add them to the new grammar.

Example: (Miniature) English Grammar L1

Grammar	Lexicon
$S \rightarrow NP\ VP$	$Det \rightarrow that \mid this \mid the \mid a$
$S \rightarrow Aux\ NP\ VP$	$Noun \rightarrow book \mid flight \mid meal \mid money$
$S \rightarrow VP$	$Verb \rightarrow book \mid include \mid prefer$
$NP \rightarrow Pronoun$	$Pronoun \rightarrow I \mid she \mid me$
$NP \rightarrow Proper\text{-}Noun$	$Proper\text{-}Noun \rightarrow Houston \mid NWA$
$NP \rightarrow Det\ Nominal$	$Aux \rightarrow does$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid to \mid on \mid near \mid through$
$Nominal \rightarrow Nominal\ Noun$	
$Nominal \rightarrow Nominal\ PP$	
$VP \rightarrow Verb$	
$VP \rightarrow Verb\ NP$	
$VP \rightarrow Verb\ NP\ PP$	
$VP \rightarrow Verb\ PP$	
$VP \rightarrow VP\ PP$	
$PP \rightarrow Preposition\ NP$	

Result of Conversion of L1 to CNF

$\mathcal{L}_1$ Grammar	$\mathcal{L}_1$ in CNF
$S \rightarrow NP\ VP$	$S \rightarrow NP\ VP$
$S \rightarrow Aux\ NP\ VP$	$S \rightarrow X1\ VP$
	$X1 \rightarrow Aux\ NP$
$S \rightarrow VP$	$S \rightarrow book \mid include \mid prefer$
	$S \rightarrow Verb\ NP$
	$S \rightarrow X2\ PP$
	$S \rightarrow Verb\ PP$
	$S \rightarrow VP\ PP$
$NP \rightarrow Pronoun$	$NP \rightarrow I \mid she \mid me$
$NP \rightarrow Proper\text{-}Noun$	$NP \rightarrow TWA \mid Houston$
$NP \rightarrow Det\ Nominal$	$NP \rightarrow Det\ Nominal$
$Nominal \rightarrow Noun$	$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal\ Noun$	$Nominal \rightarrow Nominal\ Noun$
$Nominal \rightarrow Nominal\ PP$	$Nominal \rightarrow Nominal\ PP$
$VP \rightarrow Verb$	$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb\ NP$	$VP \rightarrow Verb\ NP$
$VP \rightarrow Verb\ NP\ PP$	$VP \rightarrow X2\ PP$
	$X2 \rightarrow Verb\ NP$
$VP \rightarrow Verb\ PP$	$VP \rightarrow Verb\ PP$
$VP \rightarrow VP\ PP$	$VP \rightarrow VP\ PP$
$PP \rightarrow Preposition\ NP$	$PP \rightarrow Preposition\ NP$

- The original lexical rules are also carried to L1

CKY Algorithm

- With the grammar in CNF, each non-terminal node above the POS level in a parse tree will have exactly two children.
- Thus, a 2-dimensional matrix can be used to encode the tree structure.
- For a sentence of length n, work with upper-triangular portion of an (n+1)x(n+1) matrix.
- Each cell [i, j] contains the set of non-terminals that represent all constituents spanning positions i through j of the input.
  - Cell that represents the entire input resides in position [0,n]

CKY Algorithm

- Non-terminal entries: For each constituent [i, j], there is a position k, where the constituent can be split into two parts such that $i < k < j$.
  - [i,k] must lie to the left of [i, j], somewhere along row i, and
  - [k, j] must lie beneath it along column j.
- To fill in the parse table, we proceed from the left to the right in a bottom-up fashion:
  - so when we fill a cell [i, j], the cells containing the parts that could contribute to this entry (i.e., the cells to the left and the cells below) have already been filled.

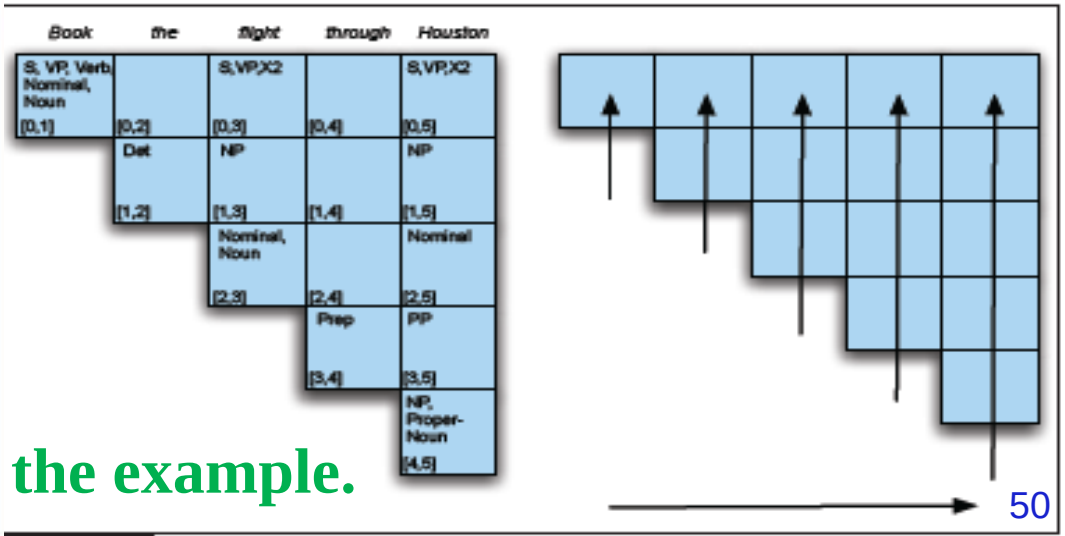
Example

$\mathcal{L}_1$ in CNF

$S \rightarrow NP\ VP$
$S \rightarrow X1\ VP$
$X1 \rightarrow Aux\ NP$
$S \rightarrow book \mid include \mid prefer$
$S \rightarrow Verb\ NP$
$S \rightarrow X2\ PP$
$S \rightarrow Verb\ PP$
$S \rightarrow VP\ PP$
$NP \rightarrow I \mid she \mid me$
$NP \rightarrow TWA \mid Houston$
$NP \rightarrow Det\ Nominal$
$Nominal \rightarrow book \mid flight \mid meal \mid money$
$Nominal \rightarrow Nominal\ Noun$
$Nominal \rightarrow Nominal\ PP$
$VP \rightarrow book \mid include \mid prefer$
$VP \rightarrow Verb\ NP$
$VP \rightarrow X2\ PP$
$X2 \rightarrow Verb\ NP$
$VP \rightarrow Verb\ PP$
$VP \rightarrow VP\ PP$
$PP \rightarrow Preposition\ NP$

Lexicon

$Det \rightarrow that \mid this \mid the \mid a$
$Noun \rightarrow book \mid flight \mid meal \mid money$
$Verb \rightarrow book \mid include \mid prefer$
$Pronoun \rightarrow I \mid she \mid me$
$Proper\text{-}Noun \rightarrow Houston \mid NWA$
$Aux \rightarrow does$
$Preposition \rightarrow from \mid to \mid on \mid near \mid through$



Explain on the example.

The CKY Algorithm

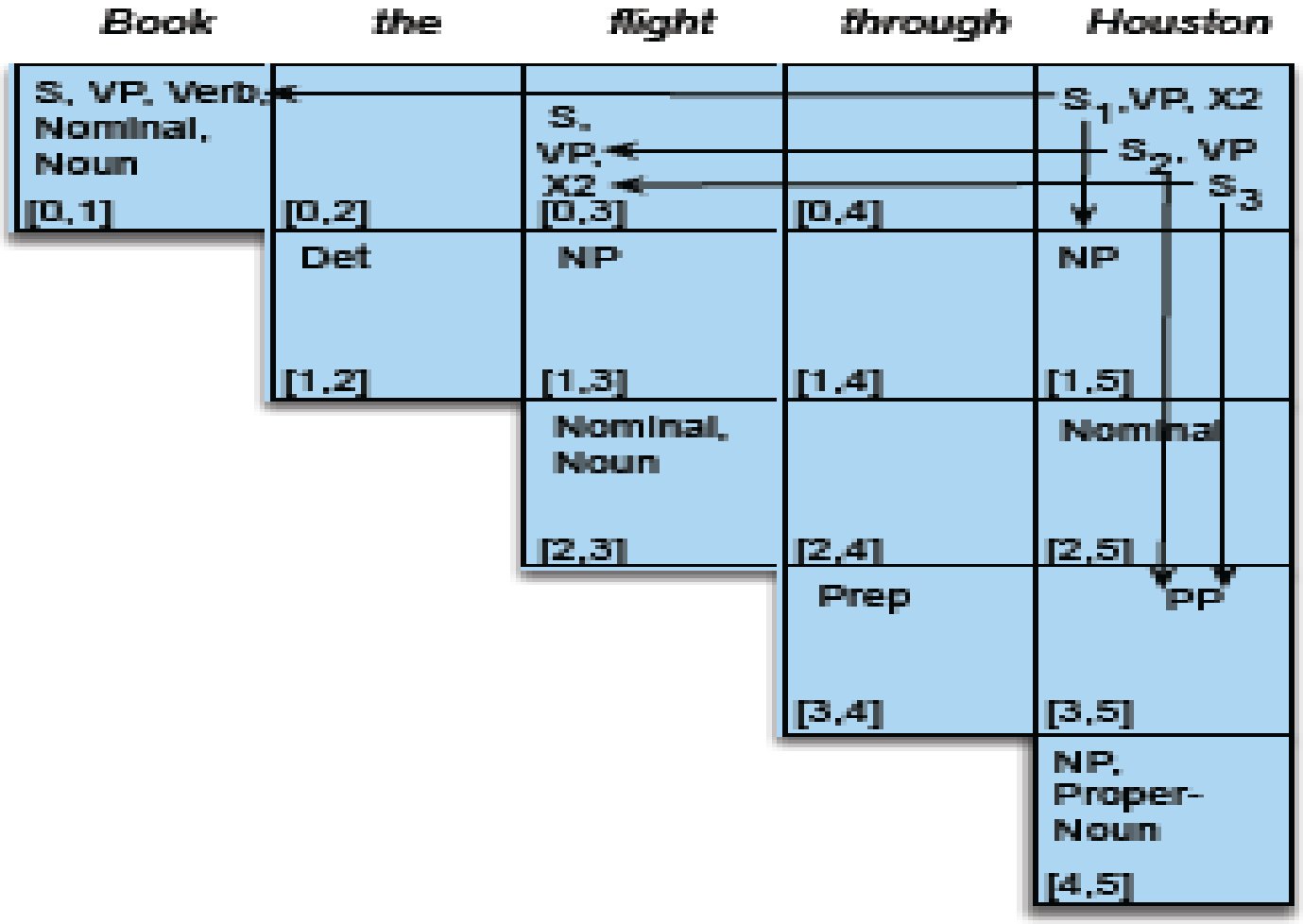
```
function CKY-PARSE(words, grammar) returns table

  for j ← from 1 to LENGTH(words) do
      for all {A | A → words[j] ∈ grammar}
              table[j − 1, j] ← table[j − 1, j] ∪ A
      for i ← from j − 2 down to 0 do
          for k ← i + 1 to j − 1 do
              for all {A | A → BC ∈ grammar and B ∈ table[i,k] and C ∈ table[k, j]}
                      table[i,j] ← table[i,j] ∪ A
```

The CKY Algorithm

- The CKY algorithm functions as a **recognizer**, not a parser:
  - it can tell whether a valid parse exists for a given sentence based on whether or not if finds an S in cell [0, n], but
  - it cannot provide the derivation, which is the actual task for a parser.
- To turn it into a parser capable of returning all possible parses, we need to make two simple changes to the algorithm:
  1. Pair each non-terminal with pointers to the table entries from which it was derived,
  2. permit multiple versions of the same non-terminal to be entered into the table.
- Then we can choose an S from cell [0, n] and recursively retrieve its component constituents from the table.

CKY Algorithm Parse Trees



CKY in Practice

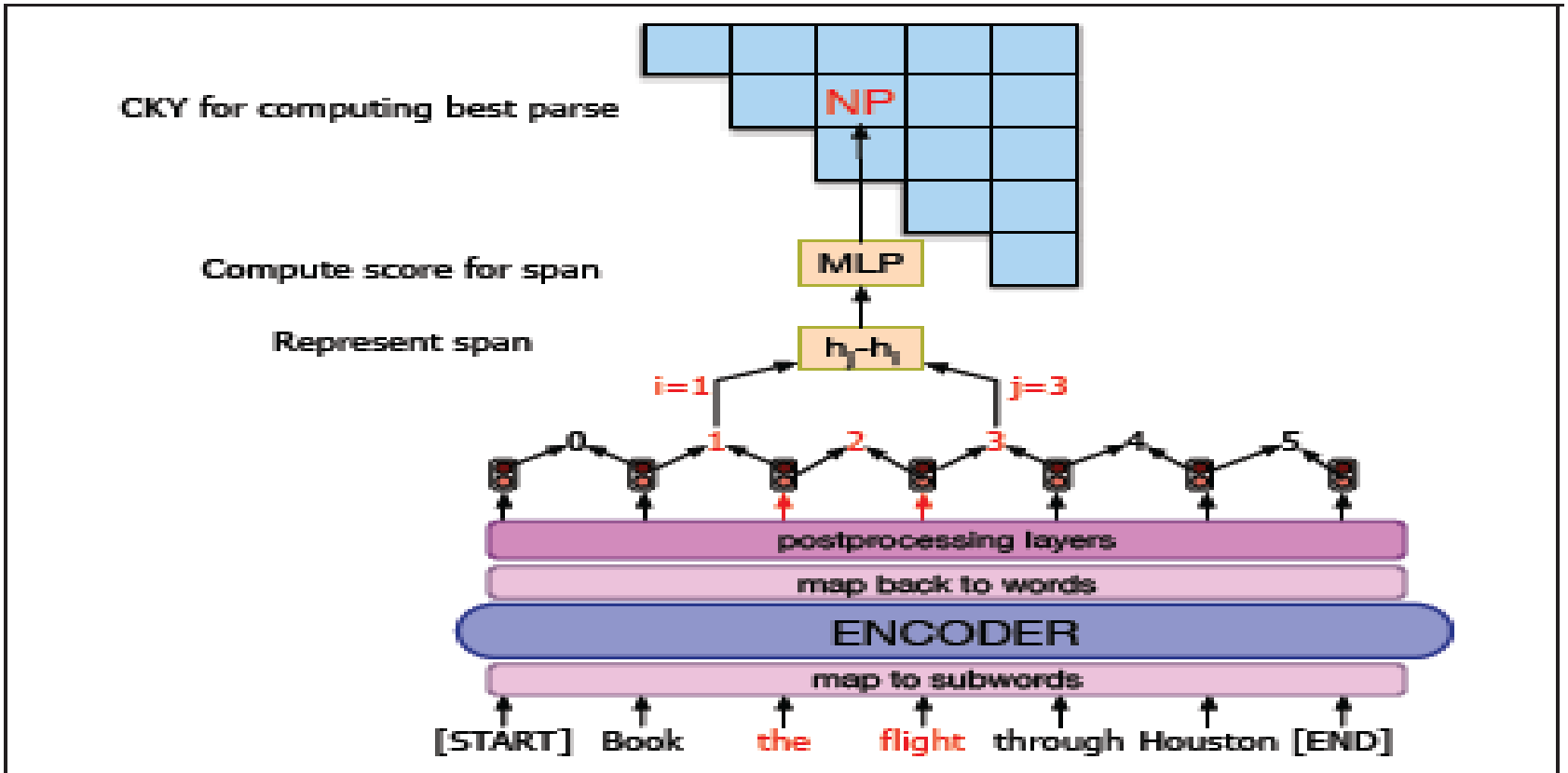
- Note that while the restriction to CNF does not pose a problem theoretically, it does pose some non-trivial problems in practice.
  - The returned CNF trees may not be consistent with the original grammar built by the grammar developers, and
  - It will complicate any syntax-driven approach to semantic analysis.
- Approach around these problems: keep enough information around to transform the trees back to the original grammar as a post-processing step of the parse. (Left as exercise.)

Span-Based Neural Constituency Parsing

- The CKY parsing algorithm we've seen enumerates all the possible parse trees for a sentence, BUT: it doesn't tell which parse is the correct one! (I.e., it **doesn't disambiguate** among the possible parses.)
- To solve this problem, can use a simple neural extension of the CKY algorithm.
- Intuition of such parsing algorithms (often called **span-based constituency parsing**, or **neural CKY**):
  - *train a neural classifier to assign a score to each constituent, then*
  - *use a modified version of CKY to combine these constituent scores to find the best-scoring parse tree.*
- The parser **learns** to map a span of words to a constituent, and,
- Like CKY, it hierarchically combines larger and larger spans to build the parse-tree bottom-up.
- Unlike CKY, It doesn't use the hand-written grammar to constrain what constituents can be combined, **instead** just relying on the learned neural representations of spans to encode likely combinations.

Computing Scores for a Span

- Consider the constituent (called a *span*) between positions *i* and *j* with non-terminal symbol label *l*.
- We'll build a system to assign a score *s(i, j, l)* to this constituent span.
- Here is a simplified outline of computing the span score for the span *the flight* with the *label NP*:

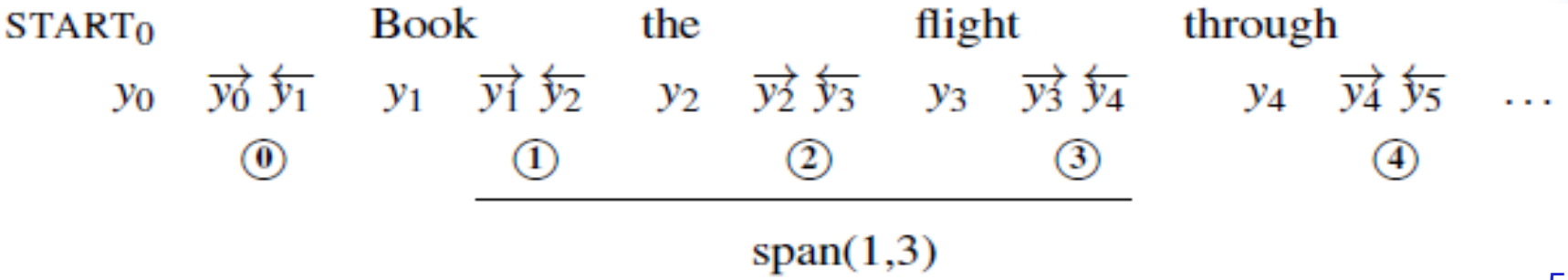


Computing Scores for a Span

- Input word tokens embedded using a pretrained LM like BERT.
- Because BERT operates on the level of subword (wordpiece) tokens rather than words, first need to convert the BERT outputs to word representations.
  - Standard way of doing this: use the first subword unit as the representation for the entire word. (Other approaches possible)
- Embeddings can then be passed through some postprocessing layers, for example, 8 Transformer layers.
- Resulting word encoder outputs y_t are then used to compute a span score.
  - First, map the word encodings (indexed by word positions) to span encodings (indexed by fenceposts), by representing each fencepost with two separate values;
  - Intuition: a span endpoint to the right of a word represents different information than a span endpoint to the left of a word.

Computing Scores for a Span

- Convert each word output y_t into a (leftward-pointing) value for spans ending at this fencepost, $\overleftarrow{y_t}$, and a (rightward-pointing) value $\overrightarrow{y_t}$ for spans beginning at this fencepost, by splitting y_t into two halves.
  - Each span then stretches from one double-vector fencepost to another,
  - Example: *the flight*, which is span(1,3):



Computing Scores for a Span

- A traditional way to represent a span: take the difference between the embeddings of its start and end, i.e., representing *span(i, j)* by subtracting the embedding of *i* from the embedding of *j*.
- Here we represent a span by concatenating the difference of each of its fencepost components: $v(i,j) = [\overrightarrow{y_j} - \overrightarrow{y_i} \; ; \; \overleftarrow{y_{j+1}} - \overleftarrow{y_{i+1}}]$
- The span vector v is then passed through an MLP span classifier, with two fully connected layers and one ReLU activation function, whose output dimensionality is the number of possible non-terminal labels:
  - s(i, j, .) = $\mathbf{W}_2$ ReLU(LayerNorm($\mathbf{W}_1$ **v**(i, j)))
- The MLP then outputs a score for each possible non-terminal.

Integrating Span Scores into a Parse

- Now we have a score for each labeled constituent span $s(i, j, l)$.
- But we need a score for an entire parse tree.
- Formally a tree T is represented as a set of $|T|$ such labeled spans, with the tth span starting at position i_t and ending at position j_t, with label l_t :

$$T = \{(i_t, j_t, l_t) : t = 1, \ldots, |T|\}$$

- Thus once we have a score for each span, the parser can compute a score $s(T)$ for the whole tree simply by summing over the scores of its constituent spans: $s(T) = \sum_{(i,j,l) \in T} s(i, j, l)$
- And we can choose the final parse tree as the tree with the maximum score: $\hat{T} = \underset{T}{\text{argmax}} \; s(T)$

Integrating Span Scores into a Parse

- Method to produce the most likely parse: use a variant of the CKY algorithm to find the full parse.
- We define $s_{best}(i, j)$ as the score of the best subtree spanning (i, j).
  - For spans of length one, we choose the best label:

$$s_{best}(i, i+1) = \max_{l} s(i, i+1, l)$$

  - For other spans (i, j), the recursion is:

$$s_{best}(i, j) = \max_{l} s(i, j, l) + \max_{k}[s_{best}(i, k) + s_{best}(k, j)]$$

- The role of the grammar in classical parsing is to help constrain possible combinations of constituents.
- By contrast, the neural model seems to learn these kinds of contextual constraints during its mapping from spans to non-terminals.

Evaluating Parsers

- The standard tool for evaluating parsers that assign a single parse tree to a sentence is the PARSEVAL metric.
- It measures how much the constituents in the hypothesis parse tree look like the constituents in a hand-labeled, reference parse.
- A human-labeled reference ("gold standard") parse tree is required for each sentence in the test set.
- Reference parses usually drawn from a treebank like the Penn Treebank.
- A constituent in a hypothesis parse C_h of a sentence s is labeled correct if there is a constituent in the reference parse C_r with the same starting point, ending point, and non-terminal symbol.

Evaluating Parsers

- We can then measure the precision and recall

$$\textbf{\textit{labeled recall}} = \frac{\text{\# of correct constituents in hypothesis parse of s}}{\text{\# of total constituents in reference parse of s}}$$

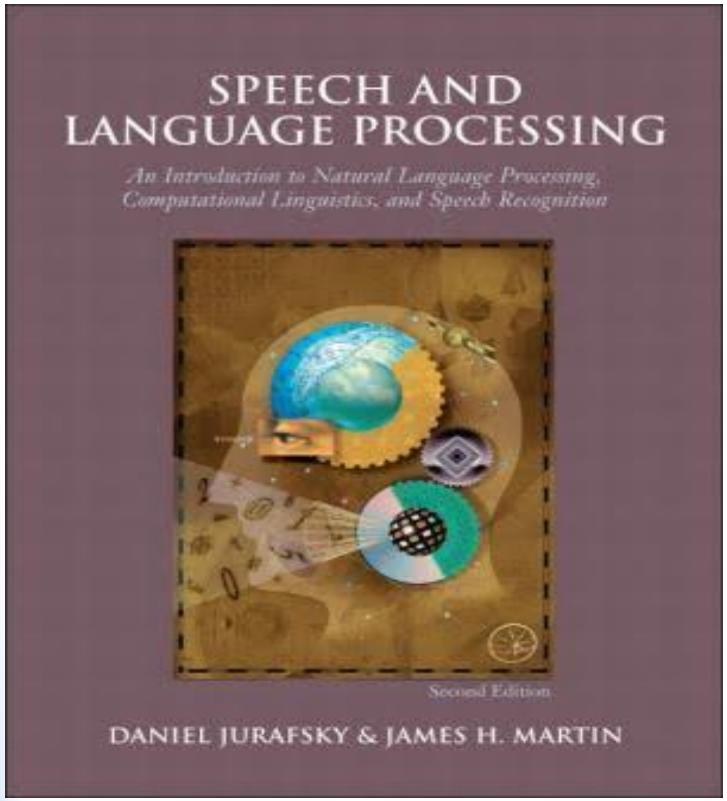
$$\textbf{\textit{labeled precision}} = \frac{\text{\# of correct constituents in hypothesis parse of s}}{\text{\# of total constituents in hypothesis parse of s}}$$

- As usual, we often report a combination of the two, F1:

$$F_1 = \frac{2PR}{P+R}$$

Textbook

Jurafsky, D. and Martin, J.H. (2024) *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models* (3rd ed. draft), Prentice Hall.
https://web.stanford.edu/~jurafsky/slp3/ed3bookaug20_2024.pdf



The draft version of the 3rd edition will be used; it has been updated to include more recent topics such as Vector Semantics and Embeddings, ANNs and Deep Learning for NLP, Transformers, etc.

This course **slides** are largely based on this textbook and its authors' slides with light customisations whenever seen fit.